Prompt 合集 · 30 条可直接改用的模板

> 配套《后端如何在一个月内转行大模型应用开发》第六部分。 每条都给出完整模板 + 一句"为什么这么写"。 用法：把 `{...}` 里的占位符换成你的内容，其余直接复制。

一、基础控制类（先让模型听话）

01 角色设定 + 硬约束

```
你是一名 {角色}，有 {年限} 年经验，服务于 {业务场景}。

回答时严格遵守：
1. 只回答与 {领域} 相关的问题，其他一律说明无法回答
2. 不确定的地方直接说"我不确定"，不要推测
3. 回答控制在 {字数} 字以内

问题：{question}
```

> 要点：**约束写成带编号的清单**，比一段话有效得多。模型对列表形式的规则更敏感。

02 结构化输出（JSON）

```
从下面文本中提取信息，严格按 JSON 输出，不要代码块包裹，不要任何解释。

字段：
- title：字符串
- tags：字符串数组，2-4 个
- summary：字符串，不超过 50 字

找不到就填 null，禁止编造。

文本：{text}
```

> 要点：最后一句"找不到填 null、禁止编造"是关键。不加这句，模型一定会给你编一个看起来很像的值。

03 拒答 / 承认不知道

回答下面的问题。如果依据不足，直接回答"依据不足，无法判断"，
并说明还缺少哪些信息。不要为了让答案完整而推测。

问题：{question}
已有信息：{context}

> 要点：告诉它"缺什么"比只说"不知道"有用，能直接引导你补数据。

04 长度与格式双控

请用不超过 {N} 字回答，格式为：
第一段 - 结论（一句话）
第二段 - 三个要点（用 - 开头）
第三段 - 下一步建议

问题：{question}

> 要点：给**结构模板**比只给字数有效，模型照着填空比自己发挥准确得多。

二、内容创作类

05 技术文章大纲

我要写一篇面向 {读者群体} 的技术文章，主题是 {主题}。

请输出：
1. 三个备选标题（分别偏干货 / 偏故事 / 偏争议）
2. 文章大纲，四级标题以内，每节标注字数建议
3. 每节的核心论点一句话
4. 你认为读者最容易卡住的是哪一节，为什么

06 标题批量生成

```
为下面这篇文章生成 10 个标题，分成三组：
- 干货型（3 个）：突出可操作性
- 悬念型（3 个）：制造信息缺口
- 数字型（4 个）：含具体数字

要求：20 字以内，不用感叹号，不出现"震惊""必看"。

文章内容：{content}
```

07 长文精简

```
把下面内容压缩到原长度的 30%，要求：
1. 保留所有结论和 data
2. 删除铺垫、重复、客套话
3. 保持原有的逻辑顺序
4. 压缩后仍要能独立读懂，不依赖原文

原文：{text}
```

08 风格模仿

```
下面是三段我写的文字，请分析其风格特征（句式长度、用词习惯、
标点偏好、是否有口头禅），然后用同样风格改写第四段。

样本 1：{s1}
样本 2：{s2}
样本 3：{s3}
待改写：{target}
```

> 要点：**先让它分析再让它写**，比直接说"用这个风格写"准确很多。

09 分层摘要

对下面内容做三层摘要：
- 一句话版：20 字以内
- 要点版：3-5 条，每条不超过 20 字
- 详细版：200 字以内，保留关键细节和 data

内容：{text}

三、代码相关

10 代码解释（面向新手）

解释下面这段代码。要求：
1. 先用一句话说它整体在干什么
2. 再逐段解释，每段说明"输入什么 / 做了什么 / 输出什么"
3. 指出其中容易踩坑的地方
4. 不要用"显然""众所周知"这类词

代码：
```{code}```

11 代码审查

审查下面代码，按严重程度分组输出：

【必须改】会导致 bug、安全问题、性能问题的
【建议改】影响可维护性的
【可选】风格优化

每条给出：问题位置、原因、修改建议（给代码）。

代码：
```{code}```

12 Bug 定位

```
我遇到一个 bug：
现象：{symptom}
期望：{expected}
实际：{actual}
相关代码：{code}
已尝试：{tried}

请按以下步骤输出：
1. 列出 3-5 个可能原因，按概率排序
2. 每个原因给出验证方法（具体命令或断点位置）
3. 给出最可能原因的修复方案
```

> 要点：让它**先列假设再验证**，而不是直接猜一个答案。这是调试 Prompt 最有效的写法。

13 单元测试生成

```
为下面的函数生成单元测试，要求：
1. 覆盖正常输入、边界值、异常输入三类
2. 边界至少包含：空值、极值、非法类型
3. 每个用例注明它在测什么
4. 用 {测试框架} 风格

函数：{code}
```

14 重构建议

```
下面这段代码工作正常，但我想改进它。请：
1. 指出当前设计的问题（如果真的有）
2. 给出重构方案，说明每个改动解决了什么
3. 评估改动风险：会不会引入回归
4. 如果这段代码其实够用了，直接说"不建议改"，不要硬找问题

代码：{code}
```

> 要点：第 4 条很重要，否则模型为了显得有用，一定会给你编一堆"问题"。

四、数据处理类

15 信息抽取

```
从下面文本中抽取字段，输出 JSON 数组，每条一个对象。

字段：{字段列表}
规则：
- 原文没有的字段填 null
- 日期统一为 YYYY-MM-DD
- 金额统一为数字，不带单位

文本：{text}
```

16 文本分类（少样本）

```
把用户反馈分类到以下类别之一：{类别列表}

示例：
输入：{sample1} → 输出：{label1}
输入：{sample2} → 输出：{label2}
输入：{sample3} → 输出：{label3}

请分类（只输出类别名，不要解释）：
输入：{user_input}
```

> 要点：示例给 3 条。太少没效果，太多浪费 token。类别之间要有明显区分度。

17 情感与意图双层判断

对下面的句子做两层判断，输出 JSON：

```
{
  "sentiment": "正面 / 中性 / 负面",
  "intent": "咨询 / 投诉 / 建议 / 闲聊",
  "urgency": 1-5,
  "reason": "一句话说明判断依据"
}
```

句子：{text}

> 要点：让它输出 `reason`，方便你事后校验判断对不对、也方便排查 bad case。

18 非结构化文本转表格

把下面这段文字整理成表格，字段为：{字段列表}。

要求：
- 输出 Markdown 表格
- 缺失值填 "-"
- 如果原文含糊无法归入某个字段，在该单元格标注"（原文未明确）"

文本：{text}

五、RAG 专用（第四周项目直接能用）

19 知识库问答（带引用 + 拒答）

```
你是严谨的知识库助手，严格依据【资料】回答。

规则：
1. 只使用【资料】中的信息，不要用自己的知识补充
2. 【资料】中找不到就回答"资料中未提及"，绝不推测
3. 每个结论后用「依据：资料N」标注来源
4. 回答简洁，不重复资料原文

===== 资料 =====
{context}
===== 资料结束 =====

问题：{question}
```

> 要点：第 3 条"标来源"既是给用户核验，也是你排查检索问题的抓手 —— 模型总标错来源，说明召回的片段本身有问题。

20 多轮查询改写（指代消解）

```
把用户的追问改写成独立完整的查询，使其脱离上下文也能被检索到。

对话历史：{history}
用户追问：{follow_up}

要求：
- 只输出改写后的查询，不要解释
- 如果追问本身已经完整，原样返回
- 把代词替换成具体实体
```

21 HyDE（假设性文档嵌入）

```
下面的问题要用向量检索去找资料，但直接检索效果不好。
请先写一段"假设性的答案文档"（200 字以内），
用这段文档去检索，比直接用问题检索更准。

注意：这只是检索用的中间产物，不是最终答案，可以包含推测。

问题：{question}
```

> 要点：HyDE 的核心就是"用答案的样子去找答案"。明确告诉模型这是中间产物，否则它会纠结准确性。

22 答案忠实度校验

```
判断下面的【答案】是否完全忠于【资料】。

输出 JSON：
{
  "faithful": true/false,
  "unsupported": ["答案中无法在资料里找到依据的句子"],
  "score": 1-5
}

【资料】{context}
【答案】{answer}
```

六、Agent / 工具调用类

23 工具选择判断

```
可用工具：{工具列表及说明}

用户请求：{request}

请判断：
1. 是否需要调用工具？（能直接回答就不调）
2. 如果需要，调用哪个，参数是什么
3. 如果需要多个，调用顺序是什么

输出 JSON：{"need_tool": bool, "calls": [{"tool": "", "args": {}}]}
```

24 任务拆解（规划）

```
把下面的目标拆解成可执行的步骤。要求：
1. 每步只做一件事
2. 标明步骤间依赖（哪步必须在哪步之后）
3. 标明每步需要什么输入、产出什么
4. 如果某步需要外部信息，注明需要什么

目标：{goal}
```

25 Function 参数抽取

```
从用户的话里抽取调用 {函数名} 所需的参数。

函数签名：{signature}
参数说明：{param_docs}

用户的话：{user_input}

输出 JSON。缺失的必填参数填 null 并列出，
同时生成一句向用户追问这些参数的话。
```

七、调试与优化类

26 Prompt 自我改进

下面是一个 Prompt 和它在实际使用中出现的问题。

Prompt：{prompt}
问题：{problem}

请分析原因并给出改进版本，说明每处改动针对什么问题。

27 输出一致性检查

用同一个 Prompt 跑 {N} 次，输出如下：{outputs}

请分析：
1. 哪些部分每次都一样（稳定）
2. 哪些部分波动大（不稳定）
3. 波动的原因是什么
4. 如何让输出更稳定（具体改法）

> 要点：这是排查"模型时好时坏"的标准方法，比反复试要高效。

28 思维链（CoT）

分析下面需求，按步骤输出：
第一步 - 你理解到的核心诉求
第二步 - 其中模糊、需要确认的地方
第三步 - 可能的实现思路（至少两个）
第四步 - 你推荐哪个，理由

需求：{requirement}

八、求职实战类

29 模拟面试官（带追问）

```
你是 {公司} 的 {岗位} 面试官，正在面试我。

规则：
1. 一次只问一个问题
2. 我回答后，先给出评价（好在哪、差在哪），再追问或换下一题
3. 如果遇到我答不上来的，给出提示但不要直接给答案
4. 面试结束后给出整体评价和待改进清单

我的背景：{background}
开始吧。
```

30 简历经历润色（STAR + 量化）

```
把下面这段经历改写成简历条目。

要求：
1. 每条格式：动作 + 技术/方法 + 量化结果
2. 动词开头，不用"负责""参与"这类虚词
3. 必须有数字，没有数字的帮我标注"（此处需补数据）"
4. 单条不超过两行
5. 不要编造我没做过的事，不确定就标注出来

经历：{experience}
```

> 要点：第 5 条很关键。让模型帮你润色可以，但**不能让它编经历** —— 面试官会追问细节，编的一定露馅。

附：三条通用心法

1. **约束写成编号清单**，不要写成一段话
2. **允许模型说"不知道"**，否则它一定会编
3. **让它先分析再输出**，准确率明显高于直接要答案

共 30 条 · 配套文章第六部分 · 可直接复制使用